Carleton University
School of Computer Science

COMP 3000 (WINTER 2023) OPERATING SYSTEMS ASSIGNMENT 3

Please submit the answers to the following questions in Brightspace by the due time indicated in the submission entry. There are 20 points in total + 6 bonus points (weight: 0.25).

Submit your answers as a **gzipped tarball** "*username*-comp3000-assign3.tar.gz" (where username is your MyCarletonOne username). Do NOT just submit a file of another format (e.g., txt or docx) by renaming it to .tar.gz. Unlike tutorials, assignments are graded for the correctness of the answers.

The tarball you submit **must** contain the following (applicable to all assignments):

1. A **plaintext** file containing your solutions to all questions, including explanations.
2. A README.txt file listing the contents of your submission as well as any information the TAs should know when grading your assignment. Without this file, grading will be based on the TA's understanding.
3. For each question, where applicable, a C file for your modified version of the original file. This should include all required changes for that question. This way, you can avoid something wrong with one question affecting another question.
4. Diff files showing the modifications, by comparing each submitted C file above and the original: for example, `diff -c 3000original.c 3000modified.c > 3000.diff`. Avoid moving around or changing existing code (unless necessary) which may be distracting.

For this assignment, you need to submit code for question 2 and question 6 in part 1, and question 7.d if you choose to answer it.

You can use this command to create the tarball: `tar zcvf username-comp3000-assign3.tar.gz your_assignment_directory`. ****Don't forget to include your plaintext file!!****

No other formats will be accepted. Submitting in another format will likely result in your assignment not being graded and you receiving no marks for this assignment. In particular, do not submit an MS Word, OpenOffice, or PDF file as your answers document!

As a student learning operating system, you are expected to be able to handle file transfer properly using necessary commands such as `ssh`, `scp` or `rsync`. Otherwise (which happens), **please set aside time by completing your assignment enough long before the due time**, then you may ask a TA, the instructor, or another student for any file transfer issue. Then, you can keep working on it and **override your submission** by uploading a new tarball before it is due.

For your convenience, here are a few commands (among many others) to download a file from your Openstack VM. You need to run the command on your PC, not your Openstack VM.

Empty or corrupted tarballs may be given a grade of zero, so please **double check your submission** by downloading and extracting it.

Don't forget to include what outside resources you used to complete each of your answers, including other students, and web resources. You do not need to list help from the instructor, TA, or information found in the textbook or man pages.

Use of any outside resources verbatim as your answer (like copy-paste or quotation) is not allowed, and will be treated as unauthorized collaboration (and reported as plagiarism).

Please do **NOT** post assignment solutions on MS Teams or Brightspace (or other platforms not used in the course such as Discord) or it will be penalized. Moreover, posting the assignment questions to any external forums/websites is NOT permitted and will also be penalized/reported.

Questions – part 1 [16 + BONUS]

The following questions (where applicable) will be based on the original 3000test.c in Tutorial 5. Remember to delete unneeded large files after completing this assignment.
You can make full use of the "`man`" pages.
Definition: a sparse file is a file with "holes", which means not all data blocks are allocated (blocks containing just 0's will not be allocated until written to; the corresponding data pointers in the inode just point to NULL, i.e., 0).
Create a sparse file (file with "holes") with: `truncate -s 256M myfile1`
Create another one: `dd if=/dev/zero of=myfile2 bs=1M count=1 seek=255`
Create a 3rd one: `dd if=/dev/null of=myfile3 bs=1M count=1 seek=256`
Create a 4th one: `dd if=/dev/urandom of=myfile4 bs=1M count=1`
Always examine the size of the files with `ls -ls` in this part.

1. [2] Why do `myfile2` and `myfile3` have the same logical size? Be specific and explain with numbers.

2. [3] Modify the code so that it is able to count the number of null bytes (whose value is 0) instead of "a". Which to count is controlled by how it is called. Specifically:
   (first compile it into an executable `3000test` like before)
   ```
   $ ln -s 3000test 3000test-a
   $ ln -s 3000test 3000test-0
   ```
   Then, when you provide `myfile1` above as input to `3000test-0`, you should see:
   ```
   $ ./3000test-0 myfile1
   File myfile1:
      inode 10505711
     length 268435456
   null count 268435456
   ```
   If you provide `myfile1` as input to `3000test-a`: (since it does not contain any 'a')
   ```
   $ ./3000test-a myfile1
   File myfile1:
      inode 10505711
     length 268435456
   a count 0
   ```

3. [1] In Question 2, `3000test` has traversed all the bytes in `myfile1`. If you check the physical and logical sizes using "`ls -ls`", you'll see they still remain the same as before. Why didn't the traversal cause actual allocation for those holes?

4. [1] Let's make a copy of `myfile2`:
   ```
   $ dd if=myfile2 of=myfile2.bck
   ```
   What happened to `myfile2.bck`?
   [1] Find the option to be used with `cp`, so that the `cp` command can optimize space allocation and

maximize the holes of a file:

```
$ cp [some_option] myfile2 myfile2.cp
$ cp [some_option] myfile2.bck myfile2.bck.cp
```

The physical size of both `myfile2.cp` and `myfile2.bck.cp` will be zero.

5. [1] If you use `3000test-0` to process `myfile4`, you can see that the number of null bytes is high. See the example below but your actual number will vary.

```
$ ./3000test-0 myfile4
File myfile4:
   inode 10505712
  length 1048576
 null count 7265
```

Why was `myfile4` NOT optimized to become a sparse file by the file system (or the `cp` option you found in question 4) despite the many 0's?

6. [4] Use the **unmodified** version of `3000test`. In this question, you will be able to have a concrete sense of the benefit of `mmap()`. The lines 54 – 79 are to count the occurrences of the character 'a' if the file is a regular file. Change the code to use `pread()` and avoid using `mmap()` (line 59, and correspondingly line 74) but not affect the current behavior. So, the program should still output the right number of 'a's in a regular file. You must use `pread()`.
   Caveat: pay attention to performance when using `pread()`. If it's extremely slower than the original, you may lose marks. Use of `malloc()` is recommended (otherwise you may have issues with static memory allocation).

7. You may have noticed in Tutorial 3 that, the C `struct dirent` also contains the file type information `d_type` (see `man readdir`) such as `DT_REG` for regular files. This way, you can avoid reading the actual inode with `stat()` unless needed, which may improve performance when processing a large number of files in the same directory (not our case though).
   Next, let's avoid calling `stat()` just for a change:
   a. [1] State the main reason why it will **not** be a good idea to use `d_type` in place of `st_mode` for `3000test`'s purpose from the perspective of programming/implementation.
      Do not consider portability/compatibility.

   b. [1] Even though we can use `d_type` for the file type check and avoid calling `stat()`, what information needed by `3000test` is still missing?

   c. [1] What will be a significant behavior change of `3000test` after you have switched to `struct dirent`?
      Hint: look at line 45 in the original code and recall what you learned in Tutorial 5.

   d. **[6 BONUS]** This question might become easier if your solution is based on the modification in Question 6 above.
      Now, implement the change to avoid calling `stat()`. Hint: you can mimic what is done in `3000shell` (in Tutorial 3), specifically starting from lines 158 and 164. Alternatively, you can also consider using `scandir()`.
      `3000test` must function like the original version, except that you don't need to print the missing information mentioned in 7.b.
      It is also optional to address the behavior change mentioned in 6.c above.
      You cannot call `stat()` (or any of its variants).
      Basically, you just need to replace any code involving `statbuf` and `stat()`.

Note: no partial marks are considered unless for very minor issues.

Questions – part 2 [4]

No code submission for this part.

8. [3] Recall that you created a new user called `someuser` (or a different name of your choice) in Tutorial 4. Assume that `someuser` is an attacker (or has fallen into the attacker's hands), i.e., the attacker is now logged in as `someuser`. No other access/privilege is considered. Now, based on our past discussions in this course, what are the two layers of defense technically that prevent the attacker from learning the plaintext password of the user `student`? Be specific.

9. [1] We already said there's no true atomicity as any operation (e.g., to increment a value in a memory location) is likely divisible into multiple machine instructions. Mutual exclusion is usually considered to be an equivalent. However, obviously, mutual exclusion mechanisms can also be divided into multiple machine instructions. For instance, the function to acquire a lock/mutex can be interrupted leading the lock to be acquired by two processes, defeating its purpose.
Why is mutual exclusion able to achieve atomicity? Put another way, what enables mutual exclusion mechanisms to achieve atomicity?